Spring 2026 ME/CS/ECE759 Final Project Report
University of Wisconsin-Madison

MD5 Cracker

Spiegel Xu

May 5, 2026

Abstract

This research aims to leverage massive parallel computing techniques to implement and optimize a high-throughput MD5-based password cracker on the NVIDIA RTX 4090 GPU. Although MD5 is a classic hash algorithm, when processing dictionary files at the scale of hundreds of millions of entries, traditional serial or naive parallel implementations are often limited by frequent memory allocation overhead and preprocessing bottlenecks on the CPU side.

The project initially started from a simple CUDA implementation, achieving only 14.49 MH/s. The primary performance limitations were the expensive cudaMalloc and cudaFree system calls in each batch, along with unaligned memory storage that prevented coalesced memory access. To address these issues, this project proposes and implements a series of deep architectural optimizations:

First, by introducing a Structure of Arrays (SoA) memory layout, the originally scattered string data is restructured into 16 contiguous 32-bit word arrays, enabling perfect coalesced memory access when the GPU kernel reads data.

Second, the project implements kernel fusion and instruction-level optimization. Preprocessing, computation, and result comparison are merged into a single kernel, and macro unrolling is used to completely eliminate loop branch overhead across the 64 rounds of MD5 computation.

To resolve the mismatch between the RTX 4090's massive computational power and the single-core CPU's slow preprocessing speed, this project integrates OpenMP multi-core acceleration to parallelize the SoA transposition logic. Additionally, dual-stream asynchronous pipeline technology is adopted to achieve full overlap between data transfer and computation.

Experimental results show that when processing a dictionary file of 100 million entries, the system's hash processing throughput is significantly increased from the initial 14.49 MH/s to 190.93 MH/s, achieving an end-to-end performance gain of approximately 13.1×. This demonstrates the feasibility of unleashing top-tier GPU potential by eliminating the CPU "data-feeding" bottleneck.

Link to Final Project `git` repo: https://github.com/Wwandering/MD5_ME759.git

Contents

1. Problem statement 4
2. Solution description 4
3. Overview of results. Demonstration of your project 6
4. Deliverables: 6
5. Conclusions and Future Work 7
References 8

1. General information

1. Name: Spiegel Xu
2. Email: jxu752@wisc.edu
3. Home department: CS department
4. Status: Undergraduate student
5. If applicable: No teammate
6. Choose one of the following two statements
    - I release the ME759 Final Project code as open source and under a BSD3 license for unfettered use of it by any interested party.

2. Problem statement

I have chosen to work on the default project for this course, which focuses on implementing a high-performance MD5 hash cracker. The objective is to utilize the NVIDIA CUDA platform to accelerate the brute-force search or dictionary-based attack of MD5 hashes.

3. Solution description

This section details the development process from a baseline CUDA implementation to a high-performance MD5 cracker optimized for the RTX 4090. The focus is on improving overall throughput through data structure refactoring, algorithmic instruction optimization, and the establishment of heterogeneous parallel pipelines.

2.1 Data Structure Evolution: From AoS to SoA

Baseline Design (AoS): In the baseline code, data is stored as an Array of Structures (AoS), where each thread processes an independent string object. This approach results in a large number of non-sequential memory accesses when GPU threads access memory, severely limiting DRAM bandwidth utilization.

Optimized Design (SoA): To achieve coalesced memory access, the data structure is refactored into a Structure of Arrays (SoA). On the host side, strings are pre-transposed and padded into 16 independent uint32_t buffers. Consequently, when a GPU warp executes, all threads can simultaneously read 32-bit words at the same offset, significantly improving DRAM throughput.

2.2 Algorithm Implementation and Kernel Fusion

Kernel Pruning: The baseline version splits preprocessing, hash computation, and result comparison into three separate GPU kernels. This project fuses these steps into a single kernel, md5_kernel_soa, reducing kernel launch overhead and round-trip reads/writes of intermediate results between DRAM and registers.

Instruction Optimization: The core MD5 logic is fully manually unrolled using macros (FF, GG, HH, II). By eliminating loop branch overhead and storing intermediate states (a, b, c, d) in registers, instruction-level parallelism is maximized.

Atomic Operation Mechanism: An early termination mechanism is implemented using atomicExch. Once a thread finds a matching hash, it immediately updates a global flag, allowing subsequent batch processing to terminate early.

2.3 Heterogeneous Parallel Pipeline Design
OpenMP CPU Multi-core Preprocessing: During optimization, it was observed that single-core CPU performance for SoA transposition was far slower than the consumption rate of the RTX 4090. Therefore, the omp parallel for directive is introduced to parallelize string padding and transposition tasks across all available physical cores on the server, thoroughly solving the GPU "starvation" issue.

Asynchronous Streams (CUDA Streams): A dual-stream (double buffering) pipeline model is adopted. Using cudaStream_t and cudaMemcpyAsync, the following three processes are overlapped:
Stream 1: GPU hash computation
Stream 2: CPU-to-GPU (HtoD) copy of the next batch of data
CPU (concurrently in background): OpenMP preprocessing for subsequent batches

2.4 Code Structure and Core Functions
md5_kernel_soa: The core GPU function responsible for performing MD5 transformations under the SoA layout.

main loop: Manages the data batching logic. A large batch size of 2,097,152 is adopted to ensure that all 128 Streaming Multiprocessors (SMs) of the RTX 4090 are fully saturated.

parse_hex: A high-performance hash parsing function that converts input hexadecimal strings into a byte order suitable for GPU processing.

2.5 Early Exit Mechanism and Atomic Synchronization
Detail Change: To avoid wasting computational power after the target password has already been found, an early exit mechanism based on atomicExch is introduced.

Optimization Solution: Inside the kernel, once a thread successfully matches the hash, it uses an atomic operation to modify a global flag. On the host side, after each stream synchronization, the flag is checked. If a match has been found, the loop is immediately exited and all subsequent asynchronous stream tasks are terminated. On average, this saves a significant amount of invalid computation time when processing large dictionaries.

2.6 Constant Memory and Register Pressure Optimization
Detail Change: The MD5 algorithm involves a large number of transformation constants (K-table) and shift parameters.

Optimization Solution: These static data are stored in the GPU's constant memory, leveraging its built-in cache mechanism to accelerate reads. At the same time, by streamlining intermediate variables within the kernel, the register pressure per thread is reduced. This improves the occupancy of the RTX 4090, allowing more thread blocks to reside concurrently in the Streaming Multiprocessors (SMs).

4. Overview of results. Demonstration of your project

The project was tested using a dictionary file containing 100,000,000 words and was conducted on a server equipped with an NVIDIA RTX 4090 GPU. The experimental results demonstrate a significant performance improvement from the initial functional implementation to the final optimized version.

Optimization Stage	Core Technical Points	Throughput (MH/s)	Performance Improvement
Baseline (Old-Style)	Basic loop, frequent malloc/free, AoS	14.49	1.0X
Intermediate (SoA)	Memory pre-allocation, SoA layout, kernel unrolling	21.59	1.49X
Final	OpenMP parallel preprocessing, asynchronous stream pipeline	190.93	13.18X

5. Deliverables. Building & running your project

The codebase delivered in this project contains the following core files, designed to demonstrate the evolution from the baseline version to the final optimized version:
md5_old.cu (Baseline Implementation):
The initial version of the code, which adopts a simple AoS (Array of Structures) layout and frequently calls cudaMalloc and cudaFree when processing each batch. This file is used to generate the baseline performance results (14.49 MH/s).

md5_openMP.cu (Final Optimized Version):
The primary deliverable of this project. It integrates SoA memory transposition, kernel fusion, dual-stream asynchronous pipeline (double buffering), and OpenMP multi-core parallel preprocessing. This version achieves a peak performance of 190.93 MH/s.

wordlist_generator.py (Input Data Source):
Generates a dictionary file containing 100,000,000 test data entries (one word per line). The MD5 hash of the last generated result is computed to verify the speed of the MD5 cracking algorithm.

REMARKS
Since I forgot to perform Git commits for every minor change during the early stages of project development, to compensate for this and to clearly demonstrate the project's progressive logic to the reviewers, I have specifically created a folder named /intermediate. This folder preserves intermediate state files (snapshots) of the project at different development stages.
By retaining these intermediate files, I aim to show the TA how the project evolved through iterative steps — from identifying bottlenecks (such as the shift from GPU compute bottlenecks to CPU preprocessing

bottlenecks) — to ultimately integrating them into a high-performance version. This documents my complete thought process, from functional implementation to deep optimization of RTX 4090 performance.

HOW TO COMPILE
Final Version:
nvcc -O3 -Xcompiler -fopenmp md5_openMP.cu -o md5_openMP
./md5_openMP wordlist.txt XXXXXX(the MD5 of the last word generated by the wordlist_generator.py)

Original Version
nvcc -O3 -md5_old -o md5_old
./md5_old wordlist.txt XXXXXX(the MD5 of the last word generated by the wordlist_generator.py)

wordlist_generator.py:
python wordlist_generator.py

6. Conclusions and Future Work

5.1 Lessons Learned and Project Highlights
The primary highlight of this project is the successful optimization of a standard MD5 cracker, achieving a 13.18x speedup (from 14.49 MH/s to 190.93 MH/s) on an NVIDIA RTX 4090. The most critical lesson learned is that in high-performance computing, the GPU's raw computational power is often secondary to the efficiency of the data supply chain. Initial bottlenecks were rooted in host-side serial processing and inefficient memory layouts. By shifting focus from pure kernel optimization to holistic system-wide throughput, I was able to saturate the GPU's streaming multiprocessors more effectively.

5.2 Future Work
While the current implementation is highly performant, there is still room for improvement:
Zero-copy and mmap

Multi-GPU Scaling

5.3 Leveraging ME759 Material
CUDA Programming
Memory Management
OpenMP

References

[1] GPU-accelerated hash cracker with Rust and CUDA
Posted Dec 30, 2024 Updated Dec 31, 2024By **Yoray Herzberg**
GPU-accelerated hash cracker with Rust and CUDA | Vaktibabat

[2] https://www.rfc-editor.org/rfc/rfc1321